Interaktywna strona o zdarzeniach JavaScript

Zadanie praktyczne • click, input, change i submit

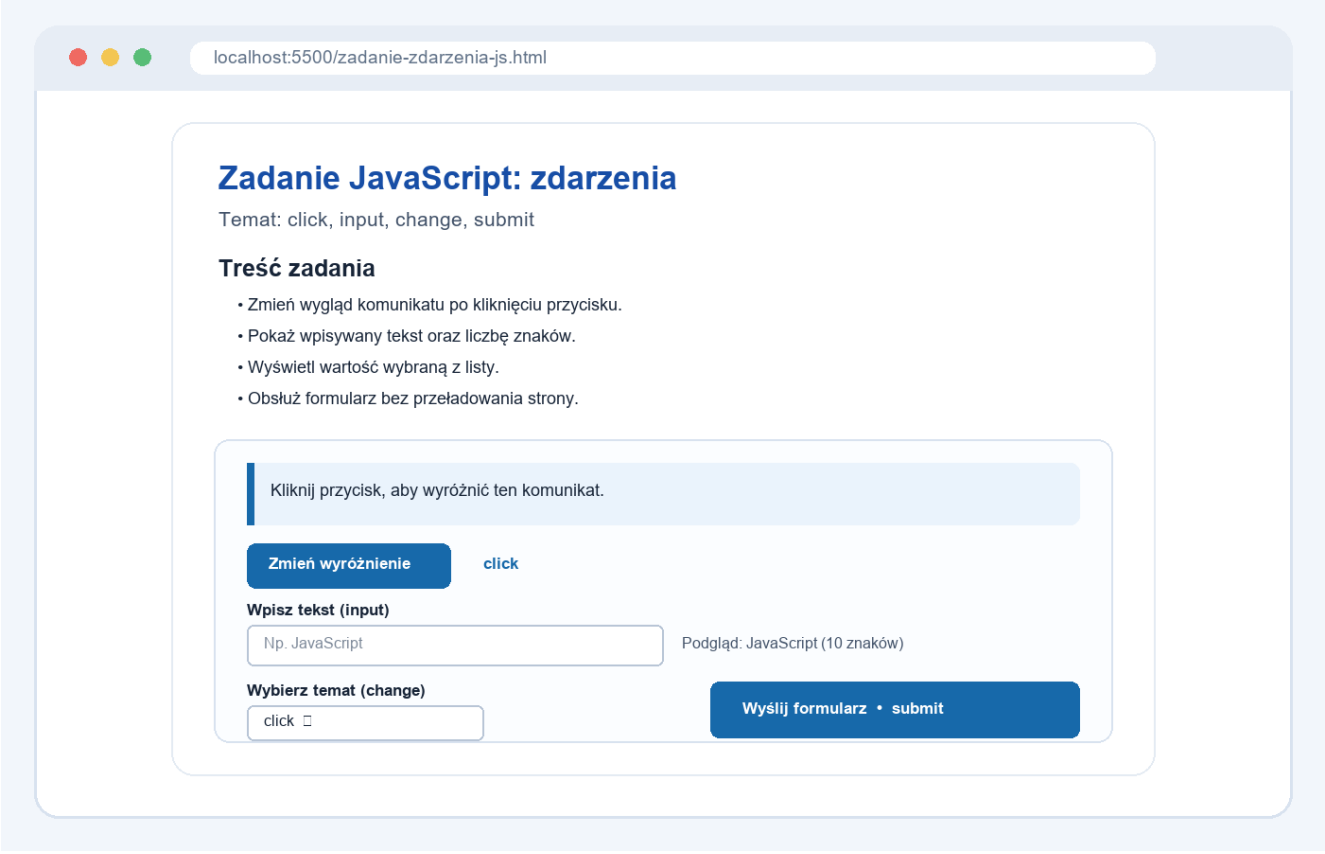
Zbuduj prostą stronę edukacyjną, na której użytkownik może wypróbować cztery zdarzenia JavaScript. Strona ma zawierać krótką treść zadania, interaktywną demonstrację oraz osobny arkusz CSS oparty na stylach poprzedniego projektu.

Efekt końcowy

Gotowy projekt ma składać się z dwóch plików:

- zadanie-zdarzenia-js.html — struktura strony i kod JavaScript.
- style-zadanie.css — wszystkie style CSS.

Po otwarciu pliku HTML w przeglądarce każda z czterech interakcji powinna działać.



Ilustracja 1. Makieta strony docelowej. Podgląd tekstu pokazuje przykładowy stan po wpisaniu wartości.

Co użytkownik powinien móc zrobić

- Kliknąć przycisk i włączyć albo wyłączyć wyróżnienie komunikatu.
- Wpisać tekst i zobaczyć podgląd oraz liczbę znaków aktualizowane podczas pisania.
- Wybrać zdarzenie z listy i zobaczyć nazwę wybranej opcji.
- Wpisać imię, wysłać formularz i otrzymać powitanie bez przeładowania strony.

Część pierwsza: przygotuj pliki i stronę

Zapisz oba pliki w tym samym folderze, aby przeglądarka mogła odnaleźć arkusz CSS.

1. Utwórz strukturę HTML

1. Utwórz dokument HTML5. Dodaj deklarację <!doctype html>, ustaw **lang="pl"**, kodowanie UTF-8 oraz meta viewport.

2. Nadaj stronie tytuł. W elemencie <title> wpisz: Zadanie JavaScript: zdarzenia.

3. Podłącz arkusz stylów w sekcji <head>: **<link rel="stylesheet" href="style-zadanie.css">**.

4. W elemencie <main> umieść nagłówek h1, temat oraz sekcje z treścią zadania i demonstracją.

2. Dodaj treść zadania

W sekcji „Treść zadania” umieść listę punktowaną, która wyjaśnia, że użytkownik ma:

- obsłużyć kliknięcie przycisku i przełączać klasę **highlight** na komunikacie;
- wyświetlać tekst i liczbę znaków na bieżąco podczas pisania;
- pokazywać nową wartość po zmianie wyboru z listy;
- obsłużyć wysłanie formularza, zatrzymać odświeżenie i wyświetlić powitanie.

3. Zbuduj obszar demonstracyjny

Komunikat i przycisk. Dodaj paragraf o id="message" oraz przycisk type="button" o id="highlightButton".

Pole tekstowe. Dodaj etykietę label połączoną przez for z polem id="nameInput". Dodaj elementy o id="typedText" i id="charCount" do podglądu i licznika.

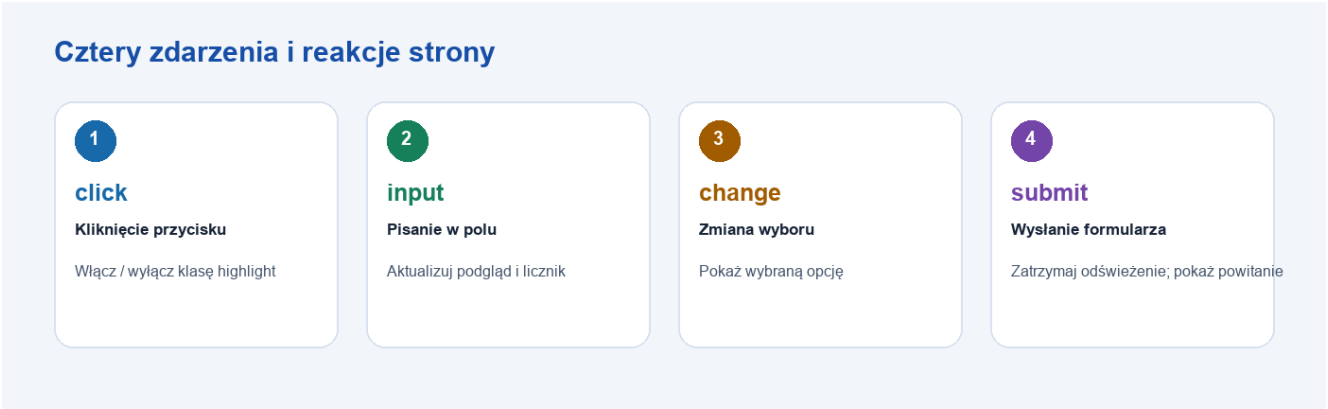
Lista wyboru. Dodaj select o id="topicSelect". Umieść opcję pustą i opcje click, input, change, submit. Dodaj paragraf id="selectionMessage".

Formularz. Dodaj form o id="greetingForm", wymagane pole id="formName", przycisk submit oraz paragraf id="formMessage" na wynik.

4. Utwórz osobny plik CSS

Skopiuj do style-zadanie.css reguły z poprzedniego projektu: :root, body, main, h1, .actions, button, button:hover, #message, .highlight, code i .note. Zachowaj wskazane kolory, odstępy, zaokrąglenia i cień. Dodaj style dla section, label, input, select oraz .demo.

Pola powinny mieć czytelne obramowanie i wygodne odstępy. Obszar demonstracyjny powinien odróżniać się od białego tła strony delikatnym obramowaniem.



Ilustracja 2. Cztery zdarzenia, elementy, które je wywołują, i oczekiwana reakcja strony.

Część druga: dodaj obsługę zdarzeń

Umieść JavaScript w elemencie <script> na końcu dokumentu, przed </body>. Pobierz potrzebne elementy przez document.querySelector() i przypisz im listenery.

5. Obsłuż click

- Dodaj listener do przycisku #highlightButton.
- Po kliknięciu użyj message.classList.toggle("highlight").
- W CSS klasa .highlight powinna zmienić kolor tekstu, tło i lewą krawędź komunikatu.

6. Obsłuż input

- Dodaj listener input do pola #nameInput.
- Odczytaj event.target.value. Przypisz tekst do #typedText, a długość do #charCount.
- Gdy pole jest puste, pokaż znak zastępczy i licznik 0.

7. Obsłuż change

- Dodaj listener change do listy #topicSelect.
- Po zmianie odczytaj event.target.value i wpisz wartość do #selectionMessage.
- Dla pustej opcji pokaż informację, że nie wybrano tematu.

8. Obsłuż submit

- Dodaj listener submit do formularza #greetingForm.
- Wywołaj event.preventDefault(), aby strona się nie przeładowała.
- Odczytaj imię, usuń zbędne spacje metodą trim() i pokaż powitanie w #formMessage.

Zasady jakości i dostępności

- Każde pole formularza powinno mieć widoczną etykietę label z poprawnym for.
- Przycisk obsługujący click powinien mieć type="button", a przycisk formularza type="submit".
- Dodaj aria-live="polite" do komunikatu z wynikiem, aby ułatwić jego odczyt technologiom asystującym.
- Nie wstawiaj danych użytkownika przez innerHTML. Wyświetlaj je jako tekst przez textContent.

Sprawdzenie działania i kryteria

9. Przetestuj stronę

1. Otwórz zadanie-zdarzenia-js.html w przeglądarce i sprawdź, czy style się załadowały.
2. Kliknij przycisk kilka razy. Wyróżnienie powinno pojawiać się i znikać naprzemiennie.
3. Wpisz, zmień i usuń tekst. Podgląd i licznik powinny aktualizować się po każdym znaku.
4. Zmień opcję na liście. Tekst pod listą powinien odpowiadać nowemu wyborowi.
5. Wyślij formularz z imieniem. Powitanie ma się pojawić bez odświeżenia strony.

10. Kryteria zaliczenia

Obszar	Warunek zaliczenia
HTML	Wszystkie elementy mają wymagane id, etykiety i typy przycisków.
CSS	Style są w osobnym pliku poprawnie podłączonym do HTML.
click	Kliknięcie przełącza klasę highlight.
input	Podgląd i liczba znaków zmieniają się podczas pisania.
change	Strona pokazuje aktualnie wybraną wartość.
submit	Formularz pokazuje powitanie i nie przeładowuje dokumentu.
Porządek	Projekt składa się z dwóch plików w jednym folderze.

Pytania kontrolne

- Czym różni się input od change w polu tekstowym i na liście wyboru?
- Dlaczego przycisk obsługujący click ma type="button"?
- Co robi event.preventDefault() w obsłudze submit?
- Dlaczego wpisany tekst pokazujemy przez textContent zamiast innerHTML?

Pliki do oddania

- zadanie-zdarzenia-js.html
- style-zadanie.css

Oba pliki zapisz razem w jednym folderze. Wynikiem ma być strona podobna do makiety, z działającymi wszystkimi czterema zdarzeniami.